

RETAIL SALES GENAI ASSISTANT

Project Report
NAME- Himanshu Bharti
DSAI_ENG-WLRCPA9G-20250930

Field	Details
Project Title	Retail Sales GenAI Assistant
Dataset	FMCG Multi-Product Transactions Lifecycle
AI Model	Google Gemini 2.5 Flash
Interface	Gradio Chatbot UI
Database	SQLite — retail_sales.db
Language	Python

1. Introduction

In today's dynamic retail environment, businesses must continuously monitor and analyze sales data to make informed decisions and stay competitive. This project builds an intelligent, interactive chatbot that leverages Generative AI and advanced data analytics to provide insightful, real-time responses about retail sales performance. The chatbot serves as an intuitive interface for interrogating the sales_data, enabling users to explore trends, product performance, and promotional impacts without requiring any technical knowledge.

The system is powered by Google Gemini 2.5 Flash for natural language understanding, SQLite for structured data storage, Plotly for visualizations, and Gradio for the web-based chat interface. The dataset used is a real-world FMCG multi-product transactions lifecycle dataset containing fields such as transaction ID, product name, sales amount, promotional offers, and time periods.

2. Problem Statement

ABC Retail Store manages large volumes of sales transactions but lacks a simple way to analyze data without relying on technical teams. The project aims to design a GenAI-powered chatbot UI that:

- Acts as a conversational analytics assistant for the sales_data stored in a structured SQLite table.
- Understands and responds to natural language questions about sales trends, product performance, and promotional impacts.
- Provides guided query suggestions to prompt user exploration.
- Allows users to download detailed chat reports via a Download Report button.
- Displays query metadata including timestamps and response times for transparency.

3. Technologies Used

Technology	Role in Project
Python 3.x	Core programming language for all backend logic
Pandas	Data loading, cleaning, transformation and aggregation
NumPy	Numerical support and array operations
SQLite3	Lightweight database for storing and querying sales_data
Plotly Express	Interactive visualizations — line, bar, and pie charts
Google Gemini 2.5 Flash	Natural language to SQL conversion and insight generation
Gradio	Web-based chat interface for user interaction
OpenPyXL	Reading the Excel dataset into Pandas
Datetime / Time	Timestamp generation and response time measurement

4. System Architecture

The system follows a four-layer pipeline architecture where each layer handles a specific responsibility.

Layer	Technology	Responsibility
Data Layer	Pandas, SQLite	Load, clean, store, and query the FMCG sales dataset
Analytics Layer	Python Functions	KPI computation, question classification, SQL execution
AI Layer	Gemini 2.5 Flash	SQL generation from natural language, business insight generation
Interface Layer	Gradio	Chat UI, metadata display, report download functionality

5. Task 1 — Data Preparation and Preprocessing

5.1 Dataset and Loading

The FMCG dataset was loaded from an Excel file using `pd.read_excel()`. An initial inspection covered shape, column names, and data types. Missing values were checked using `df.isnull().sum()` and duplicate rows using `df.duplicated().sum()`, ensuring data quality before further processing.

5.2 Feature Engineering

The `transaction_month` column was converted to datetime using `pd.to_datetime()`. Three new time-based features were extracted: Year using `.dt.year`, Month using `.dt.month_name()`, and Quarter using `.dt.quarter`. These features enable time-series analysis within the chatbot.

5.3 SQLite Integration

A SQLite database file `retail_sales.db` was created using `sqlite3.connect()` with `check_same_thread=False`. The cleaned DataFrame was stored as the `sales_data` table using `df.to_sql()` with `if_exists='replace'`. A verification query confirmed successful storage.

5.4 Dataset Schema

Column	Type	Description
<code>transaction_id</code>	String	Unique transaction identifier
<code>customer_number</code>	Numeric	Unique customer identifier
<code>transaction_month</code>	Datetime	Month and year of transaction
<code>product_id</code>	String	Unique product identifier

product_name	String	Name of the product sold
unit_price	Float	Price per unit
volume_units	Integer	Units sold per transaction
sales_amount	Float	Total revenue from transaction
promo_offer	String	Promotional offer applied or not
Year / Month / Quarter	Derived	Time features extracted from transaction_month

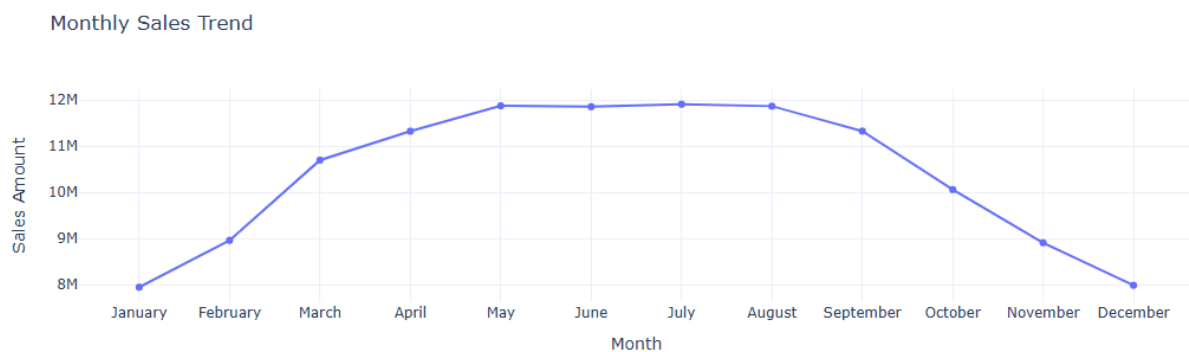
5.5 KPI Metrics Computed

Four business KPIs were computed directly from the DataFrame: Total Sales (sum of sales_amount), Total Orders (unique transaction IDs), Total Products (unique product names), and Total Customers (unique customer numbers).

6. Data Analysis and Visualizations

6.1 Monthly Sales Trend

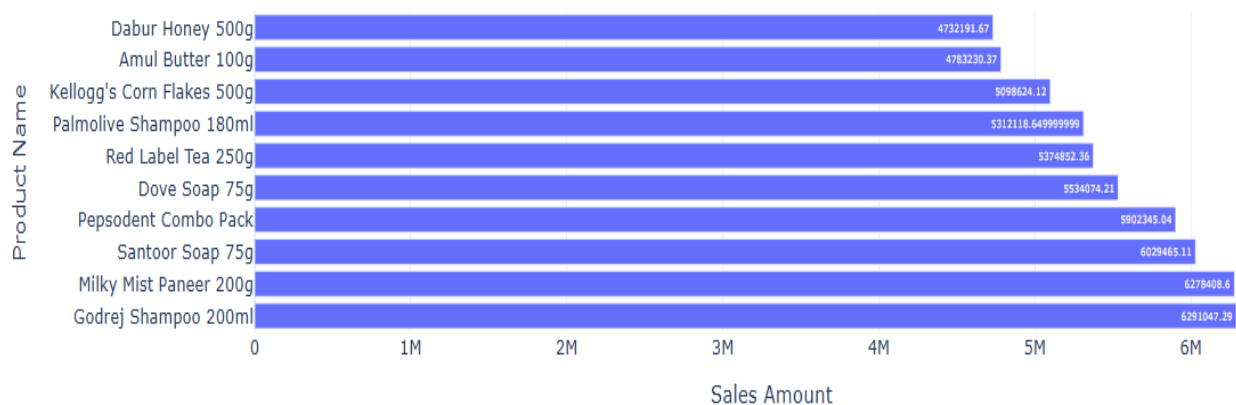
Sales were grouped by Month and summed. Months were ordered chronologically using pd.Categorical with a predefined month list. A Plotly line chart with markers was generated showing sales amount per month across the year.



6.2 Top 10 Best-Selling Products

Sales were grouped by product_name and summed, then sorted in descending order. The top 10 products were plotted as a horizontal bar chart using Plotly Express, with sales values displayed as text labels on each bar.

Top 10 Best Selling Products



6.3 Promotional vs Non-Promotional Sales

Sales were grouped by the `promo_offer` column and summed. A Plotly pie chart visualizes the revenue share between promotional and non-promotional transactions, helping the business evaluate campaign effectiveness.

Promotional vs Non-Promotional Sales



7. Task 2 — Chatbot Core Analytics Engine

7.1 Question Classification — `classify_question()`

This function detects whether the user's question is visualization-related or descriptive. It checks for keywords — `chart`, `plot`, `graph`, `trend`, `visualize` — in the lowercase version of the question. If any keyword is found, it returns 'Visualization Question'; otherwise 'Descriptive Question'.

7.2 SQL Execution — `execute_sql_query()`

This function validates that the SQL query starts with `SELECT` before executing it via `pd.read_sql()`. Results are capped at 20 rows using `.head(20)` to limit token usage. Errors are caught and returned as descriptive strings rather than crashing the system.

7.3 AI Insight Generation — `generate_ai_insights()`

This function sends the user's question and the SQL query result to Gemini, prompting it to act as an expert retail business analyst. The prompt requests key insights, trends, comparisons, and business recommendations in bullet-point format under 400 words.

7.4 Main Engine — `retail_ai_chatbot()`

This function orchestrates the full 5-step pipeline: classify the question → generate SQL → execute SQL → handle errors → generate AI insights. It is wrapped in a try-except block for robust error handling and returns a formatted response with the question type and AI insights.

8. Task 3 — GenAI Integration for Natural Language Interaction

8.1 Gemini API Setup

The Google Generative AI library was configured using `genai.configure()` with the API key. The model was loaded as `genai.GenerativeModel('gemini-2.5-flash')`. This model handles both SQL generation and business insight generation tasks.

8.2 SQL Generation — `generate_sql_query()`

A structured prompt is sent to Gemini containing the table name (`sales_data`), all 12 column names, and a set of strict rules. The rules instruct the model to return only a valid SQLite SELECT query with no markdown, no CTEs, no WITH clauses, and LIMIT 5 for ranking queries. After the response is received, markdown code blocks (````sql` and `````) are stripped. The function then uses `.upper().find('SELECT')` to extract only the SQL portion of the response.

8.3 Suggested Queries

Guided query suggestions are displayed in the Gradio UI to help users explore the data. These include: Show monthly sales trend, Which products perform best?, Analyze promotional sales, Compare quarterly performance, and Which month had lowest revenue?

9. Task 4 — User Interface Development using Gradio

9.1 Interface Layout

The UI is built using `gr.Blocks()`. It contains a Markdown title and description, a `gr.ChatInterface()` for conversations, and a Download Report section with a `gr.Button()` and `gr.File()` component. The interface launches as a local web application.

Retail Sales GenAI Assistant

AI-powered retail analytics assistant

Ask questions like:

- Show monthly sales trend
- Which products perform best?
- Analyze promotional sales
- Compare quarterly performance
- Which month had lowest revenue?

Chatbot

Which generated higher revenue: promotional or non-promotional sales?

QUESTION TYPE:
Descriptive Question

Based on the provided query result:

Retail Sales GenAI Assistant

AI-powered retail analytics assistant

Ask questions like:

- Show monthly sales trend
- Which products perform best?
- Analyze promotional sales
- Compare quarterly performance
- Which month had lowest revenue?

Chatbot

Which generated higher revenue: promotional or non-promotional sales?

QUESTION TYPE:
Descriptive Question

Based on the provided query result:

Key Insights:

- Promotional sales generated a substantial \$124,695,995.83 in revenue.

Download Chat Report

Download Report

File

Retail AI Chat Report.txt

9.4 KB

Retail Sales GenAI Assistant

AI-powered retail analytics assistant

Ask questions like:

- Show monthly sales trend
- Which products perform best?
- Analyze promotional sales
- Compare quarterly performance
- Which month had lowest revenue?

Chatbot

Direct comparison between promotional and non-promotional sales revenue cannot be made with the current dataset as the non-promotional revenue figure is unavailable.

Business Recommendations:

- **Obtain Comprehensive Data:** Prioritize collecting and integrating revenue data for non-promotional sales to enable an accurate and complete analysis of sales performance and answer the core question.
- **Analyze Profitability:** Once comprehensive revenue data is available, extend the analysis to include the gross margin and net profit for both promotional and non-promotional sales to understand the true financial impact of each sales type.
- **Strategic Optimization:** With a complete dataset, identify which sales type is more effective in driving profitable growth, and use these insights to optimize future marketing, pricing, and promotional strategies.

Timestamp:
2026-05-30 15:40:09.516578

Response Time:
7.79 seconds

Download Chat Report

Download Report

9.2 Chat Session and Metadata

The `gradio_chat()` function captures response start and end times using `time.time()` and appends the timestamp and response time in seconds to every chatbot reply. The chat history is maintained within the session through the internal `chat_history` list, which stores each turn for export.

10. Task 5 — Workflow Automation and Reporting

10.1 Automated Chat Logging — `save_chat()`

The `save_chat()` function is called after every chatbot response. It appends a dictionary containing the user's question, the assistant's response, and the current timestamp to the `chat_history` list. This list is initialized as empty at session start and grows with each query.

10.2 TXT Report Export — `export_chat_report()`

The `export_chat_report()` function writes the full `chat_history` to a file named `Retail_AI_Chat_Report.txt` using UTF-8 encoding. Each conversation turn is written in the format: `USER → ASSISTANT → TIMESTAMP`, separated by a line of 50 equals signs. If the history is empty, a default message is written instead.

11. Task 6 — Testing and Validation

End-to-end testing was performed across all system components. The table below summarizes the test cases, functions tested, and outcomes.

Test Case	Function Tested	Outcome
SQL Generation	<code>generate_sql_query()</code>	Valid SELECT query returned — Passed
SQL Execution	<code>execute_sql_query()</code>	DataFrame result returned correctly — Passed
AI Insight Generation	<code>generate_ai_insights()</code>	Bullet-point business insights returned — Passed
End-to-End Pipeline	<code>retail_ai_chatbot()</code>	Full structured response returned — Passed
Metadata Tracking	<code>chatbot_with_metadata()</code>	Timestamp and response time captured — Passed
Chat Logging	<code>save_chat()</code>	Entry appended to <code>chat_history</code> — Passed
Report Export	<code>export_chat_report()</code>	TXT file generated with full history — Passed
Gradio UI Launch	<code>chatbot_ui.launch()</code>	Interface accessible in browser — Passed

12. Challenges Faced and Solutions

Challenge	Solution
-----------	----------

Gemini returned SQL wrapped in markdown code blocks	Stripped ```sql and ``` using .replace() before using the query
AI response included text before the SELECT keyword	Used .upper().find('SELECT') to extract only the SQL portion
SQLite does not support CTEs or complex subqueries	Added strict prompt rules to avoid CTEs, WITH clauses, and nested queries
Gradio multi-threading caused SQLite conflicts	Used check_same_thread=False when creating the database connection
Month names sorted alphabetically in charts	Applied pd.Categorical with an ordered month list to enforce calendar order
Large SQL results consumed too many Gemini tokens	Capped results at 20 rows using .head(20) before passing to the AI model

13. Results and Business Insights

The system successfully processes natural language queries and returns structured business insights. The table below shows sample interactions demonstrating the chatbot's analytical capabilities.

User Query	System Behavior
Which products generated the highest sales?	Classifies as Descriptive. Generates GROUP BY product_name SQL with ORDER BY and LIMIT 5. AI returns top products with revenue insights.
Show monthly sales trend and analyze performance	Classifies as Visualization Question. Groups by Month, sums sales. AI provides seasonal trend analysis and recommendations.
Analyze promotional sales	Groups by promo_offer, compares revenue. AI evaluates promotional campaign effectiveness.
Compare quarterly performance	Groups by Quarter, sums sales. AI identifies strongest and weakest quarters with business context.

14. Advantages of the System

- No SQL knowledge required — users interact in plain English without any database expertise.
- Real-time dynamic responses — no pre-built dashboards needed; insights are generated on demand.
- Transparent metadata — every response includes timestamp and response time for accountability.
- Downloadable reports — full conversation history exported as a structured TXT file.
- Lightweight infrastructure — SQLite requires no server setup, making deployment simple.
- Guided exploration — suggested queries in the UI help users discover the system's capabilities.

15. Limitations

- AI-generated SQL is not always perfect — complex questions may produce incorrect queries.
- Keyword-based classification is simple — it does not use semantic understanding.
- No cross-session memory — chat history is cleared when the notebook is restarted.
- API key is hardcoded in the notebook, which is not suitable for production environments.
- Plotly charts are not rendered inside the Gradio interface — only available in the notebook.
- TXT export contains plain text only — no chart images or formatted tables.

16. Conclusion

The Retail Sales GenAI Assistant is a fully functional end-to-end analytics chatbot that integrates data engineering, artificial intelligence, and user interface design into a single cohesive system. By combining SQLite, Pandas, Gemini 2.5 Flash, and Gradio, the project delivers a working prototype that allows non-technical users to interact with retail sales data using plain English and receive accurate, structured business insights in real time.

The six tasks completed in this project cover the full data analytics lifecycle — from raw data ingestion and cleaning, through AI integration and chatbot development, to UI delivery and automated reporting. This project demonstrates the significant potential of Generative AI in transforming how businesses interact with their data.

17. Future Scope

- Render interactive Plotly charts directly within the Gradio UI using `gr.Plot()`.
- Replace keyword-based classification with Gemini-powered semantic intent detection.
- Add persistent cross-session memory so context is preserved between notebook runs.
- Secure API key management using environment variables or a secrets manager.
- Deploy to Hugging Face Spaces or Google Cloud Run for public access.
- Export reports as formatted PDF files with embedded charts and tables.

— End of Report —